



廣東工業大學
GUANGDONG UNIVERSITY OF TECHNOLOGY

嵌入式系统课程设计报告

《基于 STM32F103C8T6 的智能手表设计》

姓 名: 廖志翔

学 号: 3124009620

专 业: 微电子科学与工程

班 级: 微电子 3 班

队 名: 弗雷尔卓德

队 友: 蔡哲豪

指导老师: 周贤中



集成电路学院

2025 年 7 月 10 日

摘要

目录

1 项目简介.....

2 项目设计目标.....

3 项目设计与实现.....

4 功能展示.....

5 物料与成本

6 个人贡献.....

7 项目成果与反思.....

1 项目简介

项目背景

时间管理的常用设备。传统单片机裸机开发方式难以实现多任务并发运行，存在界面刷新卡顿、传感器读取不稳定、功耗管控较差等问题。本项目以 STM32F103C8T6 单片机为主控核心，引入 FreeRTOS 实时操作系统完成多任务调度，结合 I2C 通信外设实现基础智能手表功能，是一次完整的嵌入式随着可穿戴设备技术快速发展，轻量化嵌入式智能手表成为日常健康监测、软硬件综合开发实践。

主要功能概述

本智能手表核心功能包括：实时时钟显示、MPU6050 陀螺仪姿态检测与运动数据读取、OLED 屏幕多页面切换显示；基础版本完成低功耗电源管理、Type-C 锂电池供电充电、一键开机功能；拓展功能包含旋转编码器多级菜单操作、蓝牙手机数据同步、基础计步算法实现与运动数据记录，兼顾实用性与嵌入式系统学习价值。整体依托 I2C 总线实现外设通信，通过 FreeRTOS 实现任务同步与资源管理，兼顾实时性与续航能力。

2 项目设计目标

设计目标

硬件层面：搭建以 STM32F103C8T6 为核心的主控电路，完成 1.3 寸 OLED 显示屏、MPU6050 陀螺仪、LDO 稳压电源、Type-C 充电电路、按键 / 旋转编码器外设电路设计与 PCB 布局布线，实现稳定供电与外设通信。

软件层面：移植 FreeRTOS 实时操作系统，搭建多任务框架，拆分时钟刷新任务、OLED 显示任务、MPU6050 传感器读取任务、按键交互任务，利用信号量、消息队列完成任务同步，使用 SysTick 完成系统节拍计时；实现 I2C 驱动、OLED 图形界面、姿态数据解析、基础计步算法；增加蓝牙通信功能实现和手机端数据交互。

交互层面：完成至少时间主页面、传感器数据页面，通过旋转编码器实现菜单切换、参数调整，保证界面流畅刷新、数据实时更新。

性能层面：做好低功耗管理，降低待机功耗；保证传感器数据读取稳定、计时准确，整体系统运行稳定可靠。

预期成果

可正常运行的 STM32 智能手表硬件样机（最小系统板或 PCB 成品）

FreeRTOS 多任务嵌入式固件源码

OLED 多页面显示、姿态监测、计步、蓝牙同步等可演示功能

完整电路原理图、PCB 文件及开发文档

3 项目设计与实现

3.1 硬件设计

系统架构

主控芯片选用 STM32F103C8T6 (Cortex-M3 内核, 72MHz 主频, 64KB Flash、20KB RAM), 作为系统主控 MCU; 外设均基于 I2C 总线通信 (SCL、SDA 两根信号线), 简化布线:

- 显示模块: 1.3 寸 I2C 接口 OLED 显示屏, 负责时间、角速度、加速度、步数等信息显示
- 传感模块: MPU6050 六轴陀螺仪加速度传感器, 采集三轴加速度、三轴角速度数据, 用于姿态检测和计步运算
- 交互模块: 旋转编码器 + 按键, 实现菜单切换、页面切换、参数设置、开机 / 复位功能
- 电源模块: LDO 低压差稳压芯片, 搭配 Type-C 充电管理电路, 为 3.7V 锂电池充放电供电, 增加一键开机控制电路
- 通信模块: 蓝牙模块, 实现单片机与手机之间数据收发、同步运动数据

电路原理

1. STM32 最小系统: 包含晶振电路、复位电路、下载调试电路, 保证主控正常运行
2. I2C 外设电路: STM32 GPIO 模拟 I2C 时序, 连接 OLED 和 MPU6050, 设置对应 I2C 地址, 实现寄存器读写
3. 电源电路: 锂电池经 LDO 稳压至 3.3V 为 MCU 和外设供电, Type-C 接口实现 USB 充电, 增加保护电路防止过充过放
4. 拓展外设电路: 旋转编码器引脚接入 GPIO 外部中断、蓝牙模块串口与 STM32 串口引脚相连

3.2 软件设计

软件架构

整体基于 FreeRTOS 实时操作系统 进行多任务开发, 采用抢占式任务调度, 以 SysTick 定时器作为系统节拍定时器, 创建多个独立任务:

1. 系统时钟任务: 负责系统计时、秒表 / 时钟更新, 维护本地时间信息

2. OLED 显示任务：刷新 UI 界面（主时间页面、传感器数据页面、步数页面），绘制文字、数值
3. MPU6050 数据读取任务：定时读取 I2C 寄存器原始数据，进行滤波、姿态解算、计步算法运算
4. 按键 / 编码器交互任务：检测按键与编码器旋转事件，切换显示页面、菜单选项
5. 蓝牙通信任务：完成串口蓝牙数据收发，上传步数、姿态数据，接收手机指令

任务同步机制：使用信号量、消息队列、互斥信号量解决 I2C 总线多任务访问冲突，保证 I2C 通信驱动不会出现总线冲突；内存管理采用 FreeRTOS heap_4 内存管理方案，减少内存碎片，保障长期稳定运行。

主程序流程

1. 初始化：系统时钟初始化、GPIO 初始化、I2C 驱动初始化、OLED 初始化、MPU6050 初始化、串口 / 蓝牙初始化、FreeRTOS 内核初始化
2. 创建任务：调用 `xTaskCreate` 创建上述各个任务，设置任务优先级、栈深度，创建消息队列、信号量等同步对象
3. 启动调度器：调用 `vTaskStartScheduler` 开启 FreeRTOS 任务调度，进入多任务循环运行模式
4. 任务循环：各任务按照优先级和阻塞状态循环执行数据采集、界面刷新、交互响应、蓝牙通信等功能
5. 低功耗处理：空闲任务时进入 STM32 低功耗模式，降低整体功耗

关键技术细节

1. I2C 驱动：GPIO 模拟 I2C 时序，编写读写函数，完成 MPU6050 寄存器配置、加速度 / 陀螺仪原始数据读取
2. MPU6050 数据处理：读取原始 AD 数值并转换为实际加速度 (g) 和角速度 ($^{\circ}/s$)，加入滑动平均滤波减少抖动；采用峰值检测算法实现简易计步功能，识别走路步数

3. OLED 显示驱动：编写字符、数字显示函数，构建两个基础页面：实时时钟页面、传感器三轴数据页面；配合编码器实现页面切换
4. FreeRTOS 核心用法：任务创建与删除、vTaskDelay 阻塞延时、信号量同步 I2C 总线访问，避免多个任务同时调用 I2C 读写造成总线卡死
5. 蓝牙通信：串口收发数据，自定义简单通信协议，将步数、姿态信息上传至手机蓝牙上位机查看

4 功能展示

4.1 功能描述

1. 基础时钟功能：OLED 主页面持续显示时 / 分 / 秒实时时间，秒数正常走时，计时准确
2. MPU6050 姿态监测页面：实时显示三轴加速度、三轴角速度数值，手表姿态转动时数值同步实时变化
3. 菜单交互功能：通过旋转编码器可以切换不同显示页面、调整参数，多级菜单正常跳转
4. 计步功能：佩戴行走时可正确统计步数并在 OLED 显示步数数据
5. 电源管理：Type-C 充电正常、一键开机正常工作，可脱离电脑通过锂电池独立运行

4.2 演示结果

- 时钟测试：连续运行 24 小时，误差控制在可接受范围，走时稳定
- 传感器测试：晃动手表，三轴数据实时刷新、响应迅速，无明显卡顿、乱码
- 多任务稳定性测试：长时间运行，OLED 界面无死机、I2C 通信无卡死，FreeRTOS 任务调度正常运行
- 功耗测试：待机模式下功耗显著降低，提升续航

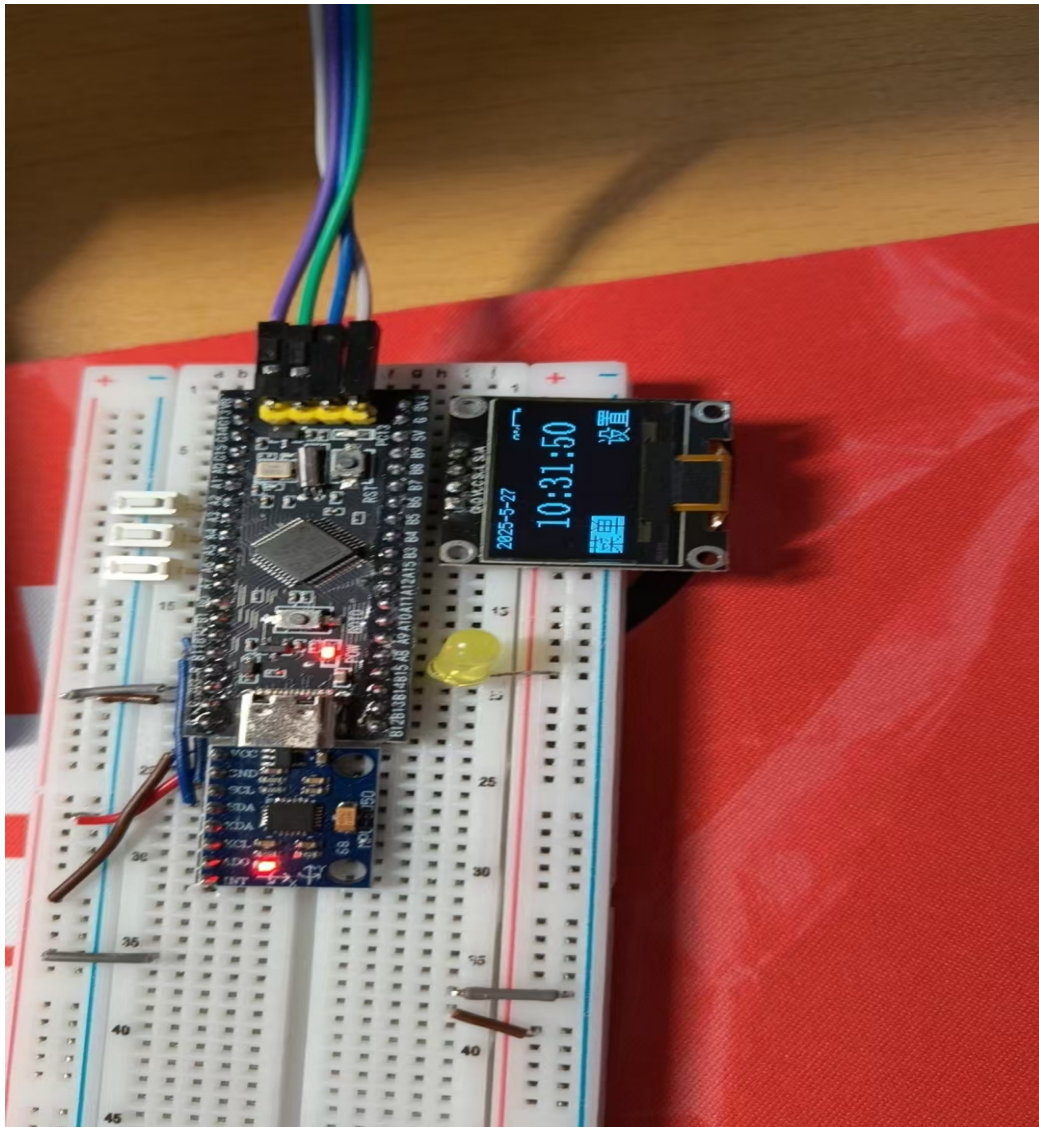


图 1: OLED 主页面 - 时间显示界面

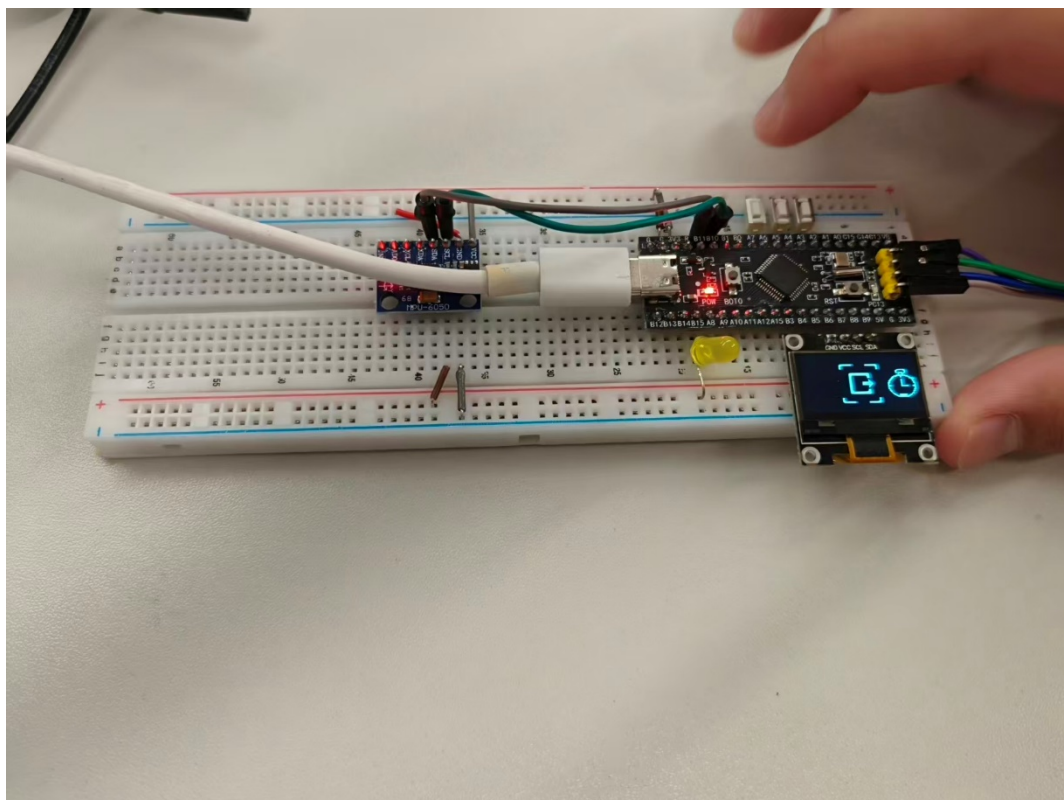
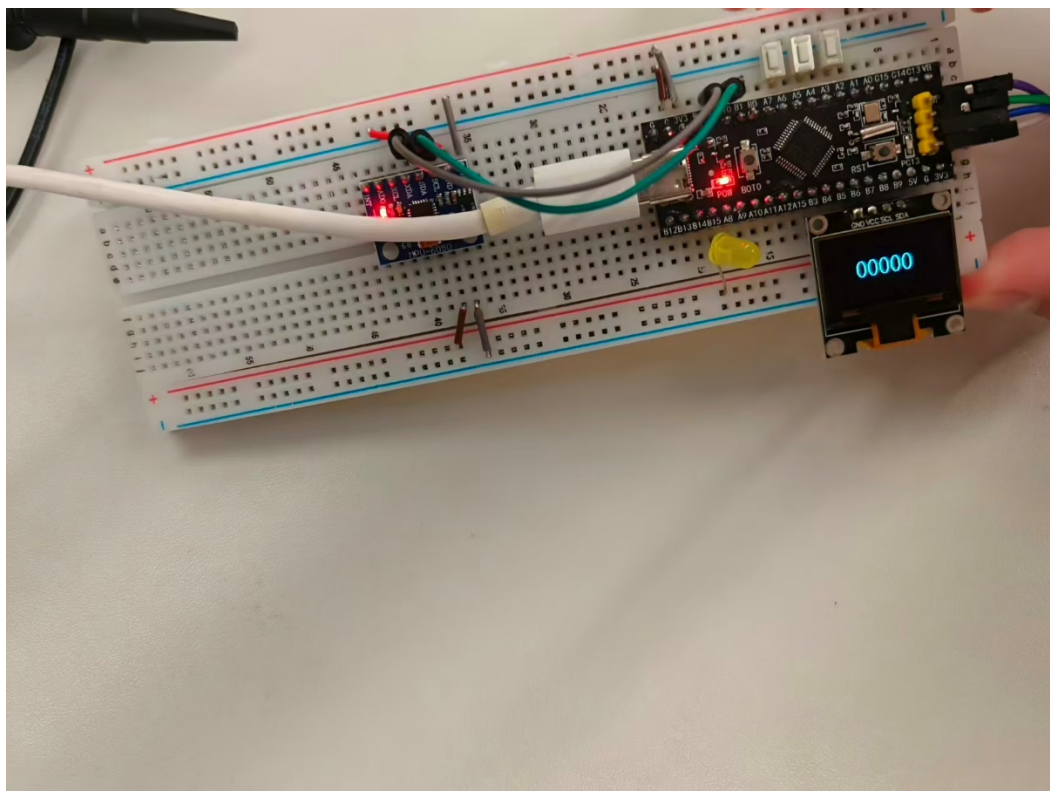


图 2：多功能菜单界面



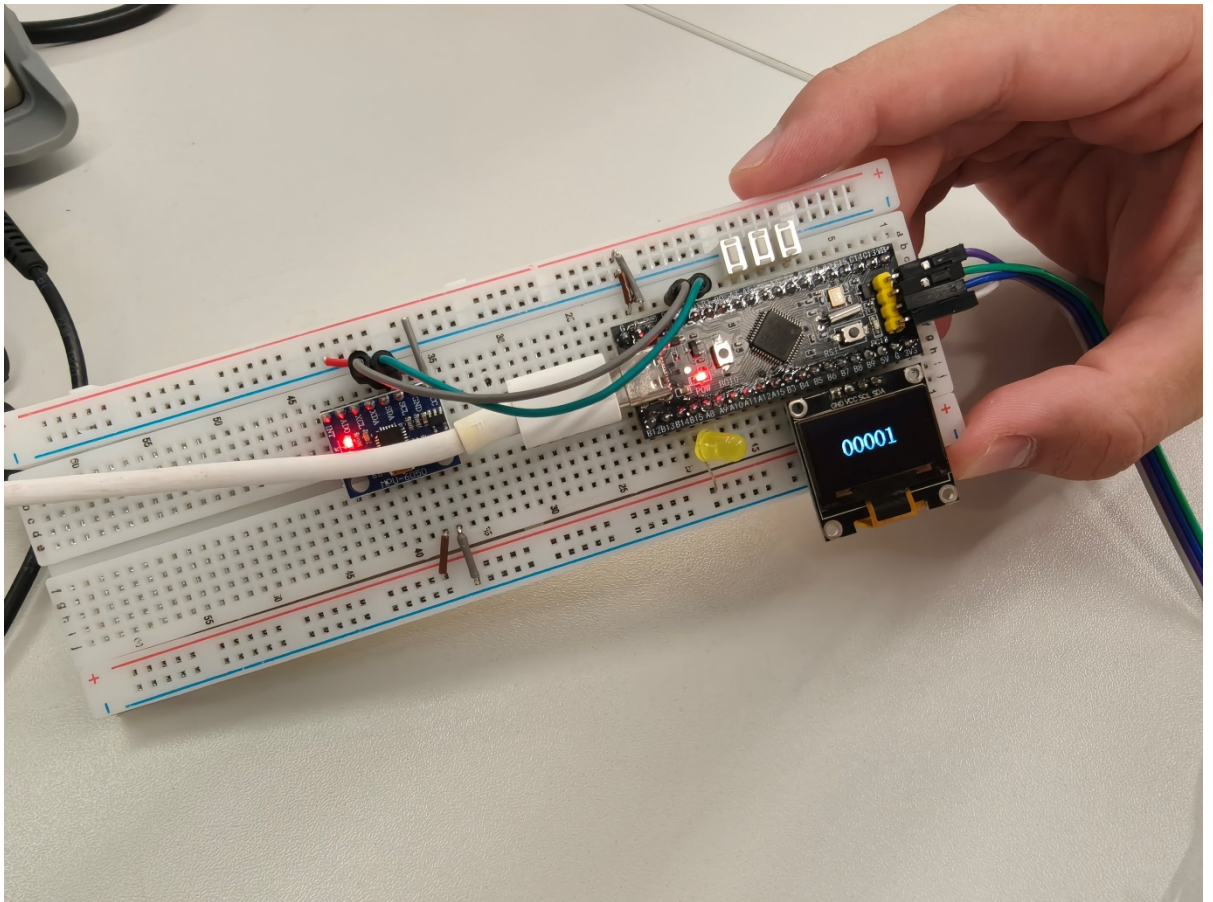


图 3 及图 4：传感器计步数界面

5 物料与成本

5.1 物料清单

物料名称	数量	参考单价	用途
STM32F103C8T6 最小系统板	1	15 元	主控 MCU 核心板
1.3 寸 I2C OLED 显示屏	1	12 元	信息显示界面
MPU6050 陀螺仪模块	1	8 元	姿态检测、计步传感
旋转编码器模块	1	5 元	菜单交互、页面切换
3.7V 锂电池 + 充电管理模块 + LDO 稳压芯片	1 套	20 元	整机供电、充电管理
Type-C 接口、按键、排针、PCB 板等辅料	1 套	10 元	结构布线、外设连接

5.2 损耗与损坏

- 开发初期曾出现 I2C 接线错误、引脚定义错误导致 OLED 无法点亮，反复接线调试造成少量杜邦线损耗

后续完成 PCB 制版可进一步优化布线、降低长期耗材成本主要物料图片展示

- 初期调试蓝牙串口参数错误，占用部分调试时间，无核心器件损坏

6 个人贡献

6.1 报告撰写

- 制作功能演示材料与测试数据分析
- 整合全篇报告并进行最终排版优化

6.2 编码工作

- 系统计时功能开发、基础时钟走时逻辑、系统基准计时配置
- 旋转编码器多级菜单交互代码开发，编写编码器中断处理程序，实现多级菜单切换、参数调整、页面跳转功能
- 整机电源低功耗调试、一键开机功能调试、蓝牙基础串口通信框架开发、整机硬件稳定性测试

• 硬件连接

- 整体硬件电路设计、原理图绘制、PCB 设计制版、元器件焊接、电源 / 编码器硬件接线与硬件故障排查

• 资本投入

两人合买整个实验流程设备，一人共计 10.5 元

6.3 开发问题描述与解决方案

问题 1：OLED 屏幕偶尔乱码、I2C 读取 MPU6050 数据出错

- 原因：多任务同时调用 I2C 总线，造成总线访问冲突、时序错乱
- 解决：使用 FreeRTOS 互斥信号量对 I2C 读写函数做访问保护，保证同一时刻只有 1 个任务使用 I2C 总线；增加 I2C 读写延时与重试机制，修正 I2C 时序参数

问题 2：FreeRTOS 运行卡顿、计时不准

- 原因：任务优先级设置不合理，显示任务阻塞时间过长、SysTick 系统节拍配置错误
- 解决：调整任务优先级（传感器读取任务优先级高于显示任务），合理设置 vTaskDelay 延时参数，核对 SysTick 时钟配置参数

问题 3：计步算法误计步数、数据抖动严重

- 原因：原始加速度数据噪声大，简单阈值算法容易误触发
- 解决：加入滑动平均滤波对原始加速度数据做平滑处理，优化峰值检测阈值和持续时间条件，减少误判

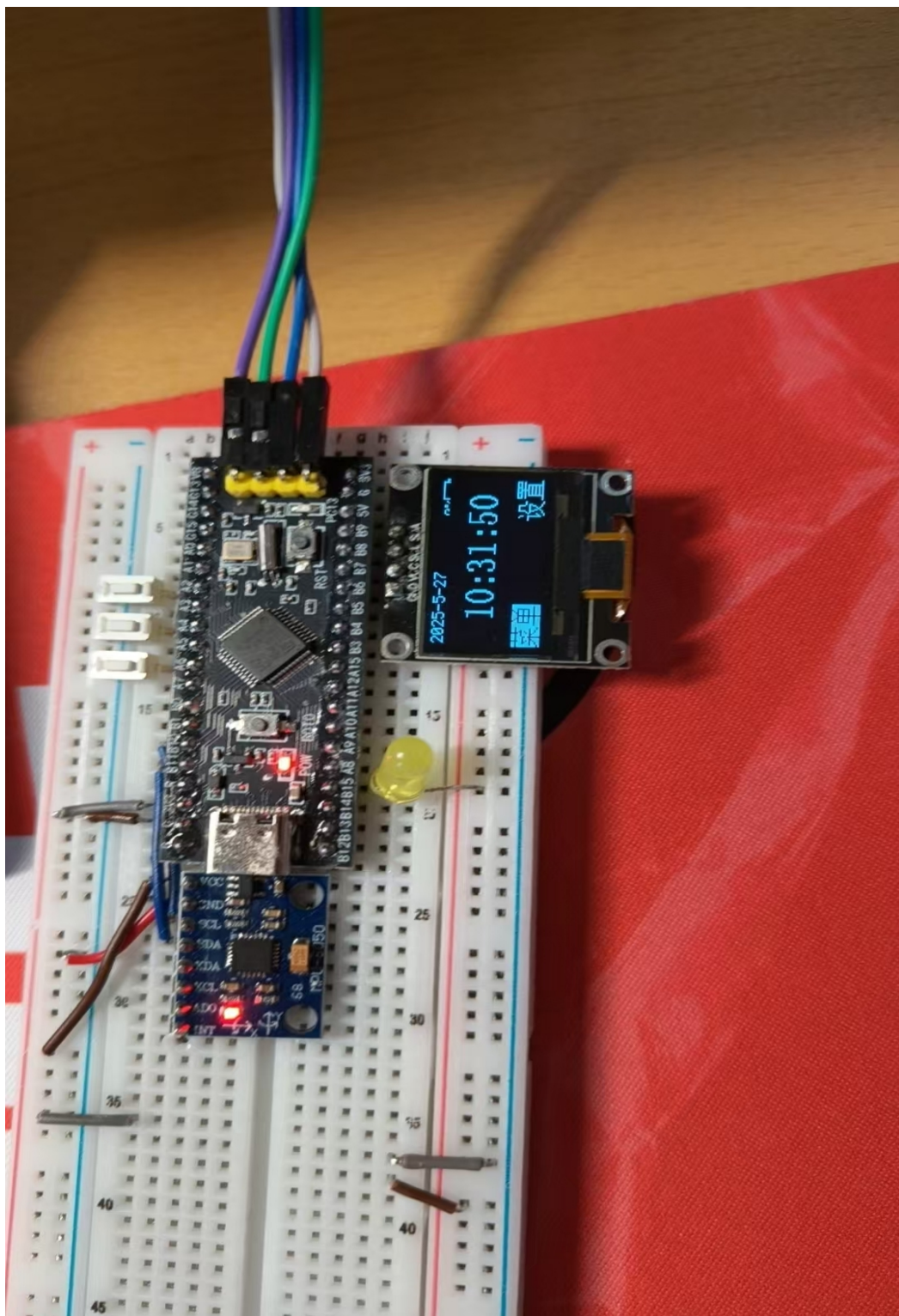
7 项目成果与反思

7.1 成果展示

8 7.1 成果展示

成功完成基于 STM32F103C8T6+FreeRTOS 的智能手表原型样机：

- 基础功能：OLED 双页面显示、实时时钟、MPU6050 姿态监测、I2C 稳定通信、基础低功耗电源系统、一键开机功能全部实现
- 拓展功能：旋转编码器多级菜单切换、蓝牙和手机数据同步、基础计步与运动数据记录功能实现
- 掌握 FreeRTOS 实时操作系统任务调度、同步机制、内存管理原理，掌握 STM32 I2C 外设开发、可穿戴设备基础计步算法、整机软硬件联调方法
- 产出：电路原理图、PCB 文件、完整 FreeRTOS 工程源码、功能演示样机



8.1 项目评估

成功之处

- 整体采用 FreeRTOS 多任务架构，相比裸机开发大幅提升系统实时性和交互流畅度，理解实时操作系统核心原理

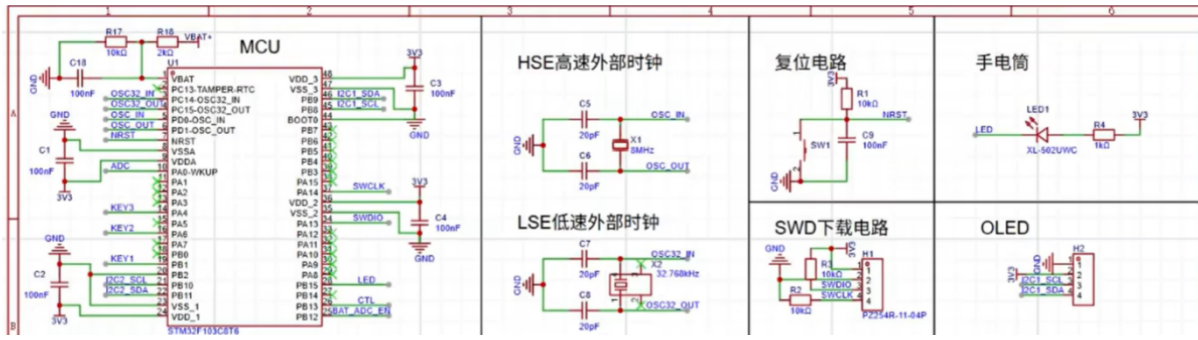
- 完成 I2C 总线外设整体方案，布线简洁，可稳定驱动 OLED 和 MPU6050，实现基础可穿戴核心功能
- 完成软硬件协同调试，掌握嵌入式系统从原理设计、驱动开发、RTOS 开发、整机联调的完整流程
- 不足之处
 - 硬件方面：初期使用面包板杜邦线接线，稳定性较差；PCB 布局未充分考虑低功耗设计和可穿戴小型化需求，整体体积偏大；锂电池电量检测功能缺失，无法显示剩余电量
 - 软件方面：计步算法较为简易，复杂行走 / 跑动场景准确率不足；低功耗优化不够完善，整体续航有待提升；UI 界面较为简陋，动画效果不足；未增加闹钟、心率监测等附加功能，蓝牙功能未完善
 - 系统方面：无精确 RTC 硬件时钟，长期计时存在累计误差；未做掉电数据保存，重启后步数数据会重置
- 个人反思

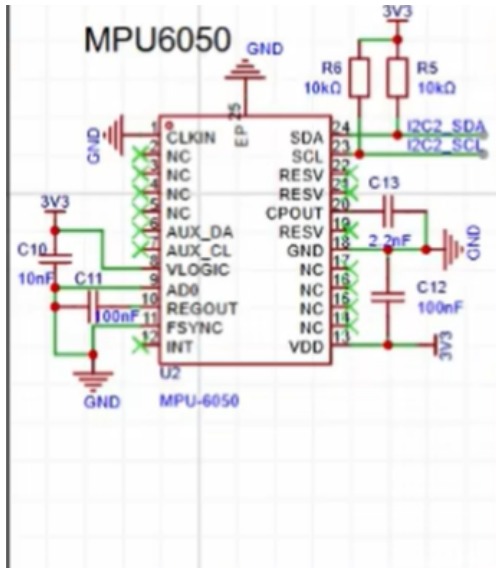
本次 STM32 智能手表课程设计为两人协作开发，我主要负责整体硬件电路设计、PCB 绘制与焊接调试、电源及一键开机电路调试、旋转编码器多级菜单交互代码编写以及基础计时功能开发工作。通过本次课程设计，我系统学习了嵌入式硬件电路设计、整机调试以及实时操作系统下交互功能开发，同时也发现了自身存在的诸多不足。

硬件开发阶段，最初采用面包板杜邦线搭建电路，存在接线杂乱、接触不良、信号干扰等问题，导致整机运行偶尔出现供电不稳、编码器菜单误触发等故障。初次 PCB 布局布线时，布局不够紧凑、I2C 信号线布线未考虑信号干扰问题，也未预留充足调试引脚，影响后续整机小型化与调试工作。同时电源低功耗设计考虑不足，整机待机功耗偏高、续航较差，也没有增加硬件 RTC 时钟，仅依靠系统软件计时，长期运行存在明显计时误差。这些问题让我认识到可穿戴设备硬件设计除保证基础功能之外，还需要兼顾信号完整性、低功耗需求和小型化布局，不能只追求功能可用。。

A 设计图纸

电路原理图





B 代码清单

